TSS2 Scenario Authoring and Custom Scene Implementation Plan

Purpose. Prepare the TSS2 Unity project so instructors can author new scenario requirements through the local scenario configuration editor and launch each scenario in its own Unity scene.

Outcome. Scenario JSON becomes the authoring source for runtime evaluation requirements, while scene, catalog, and manifest validation form a strict, reusable path for future custom scenarios.

Implementation scope

The work builds on the existing scenario-driven evaluator and preserves the current Capstone/TSS behavior. It is additive: existing TSS scenarios remain valid, deterministic VLAN and IP generation remains seed-driven, and the runtime UI continues to display scenario-authored objective headings grouped over evaluator conditions.

Area	Target behavior
Scenario data	Author device roles, hostnames, ports, trunks, unused ranges, router interfaces, hardening flags, evidence, and objective groups.
Runtime profile	Copy authored requirements into the scenario-derived profile; continue generating VLAN IDs and IP networks from profileSeed and student ID rules.
Evaluator	Read requirements from the runtime profile instead of embedding scenario-specific ports, role names, interface names, or evidence IDs.
Editor	Expose and preserve nested requirement arrays and allow valid custom scene names.
Scene launch	Load the scene named by the selected scenario after validating Build Settings and catalog compatibility.

Phase 1 Scenario requirement authoring

Complete the scenario data model and runtime profile so new scenarios can describe their evaluation contract without changing evaluator code.

- Keep authored values such as credentials, MOTD, VLAN names, evidence IDs, role bindings, interface names, port ranges, and objective headings in NetworkScenarioConfig.
- Use roleBindings to identify scenario roles and expected hostnames, including the router role and both switch roles.
- Use requiredPorts for access-port requirements, trunkRequirements for trunk interfaces and allowed VLAN names, and unusedPortRequirements for dead-VLAN shutdown ranges.
- Use routerSubinterfaceRequirements for VLAN/network mapping, address suffixes, and masks. The network base addresses and VLAN numbers remain generated by the runtime seed.
- Use explicit flags for optional grading requirements such as hardening controls, route evidence, default route, save state, AP checks, laptop checks, diagnostics, and RedWolf evidence.
- Retain defaults that reproduce the current Capstone behavior when older scenarios omit new fields.

Phase 2 Evaluator migration

Replace remaining scenario-specific constants in the evaluation path with values from TssNetworkRunProfile. Preserve condition IDs used by the HUD and results scene unless a deliberate schema migration is required.

Evaluator requirement	Scenario source
Switch roles and expected hostnames	roleBindings
Access ports and endpoint VLAN mapping	requiredPorts plus seed-derived VLAN profile
Trunk interfaces, native VLAN, allowed VLANs	trunkRequirements
Unused port ranges and dead VLAN	unusedPortRequirements
Router role and parent/WAN interfaces	routerRole, routerParentInterface, routerWanInterface
Router subinterfaces	routerSubinterfaceRequirements plus seed-derived networks
Hardening expectations	credentials, MOTD, role hostname, and hardening flags
Route and endpoint evidence	routeEvidenceCommand and evidenceRequirements
WP4 checks	AP/laptop role, switchport, evidence IDs, and requirement flags

Keep technical comparison logic in the evaluator. JSON should state what the scenario requires; the evaluator should determine whether the live state satisfies it.

Phase 3 Custom scene and catalog support

Generalize the current TSS-only restrictions so every valid scenario can reference a unique Unity scene and a compatible catalog.

- Allow any non-empty, valid sceneName in the manifest and scenario JSON.
- Require Application.CanStreamedLevelBeLoaded(sceneName) before accepting a scenario.
- Remove manifest filtering that currently requires entry.sceneName == TSS.
- Update catalog validation so the catalog sceneName must match the scenario sceneName instead of requiring the literal TSS value.
- Keep TSS_MAIN_CATALOG and the current TSS scene as the existing baseline.
- Ensure each custom scene is added to Unity Build Settings and contains the bindings required by its scenario catalog.

Phase 4 Scenario editor support

Update the local tss-scenario-config-editor site without introducing a new editor framework.

- Remove the validation message that only permits the TSS scene.
- Validate that sceneName is present and matches the project's supported scene-name format.
- Keep manifest identity fields synchronized with the scenario JSON.
- Continue recursive editing of networkScenario arrays and preserve unknown fields through Advanced JSON.
- Add concise help text explaining that the scene must exist in Build Settings and match the selected catalog.
- Provide default row templates for every new requirement array.

Files and classes to inspect

File or area	Responsibility
Assets/Scripts/TssScenarioDefinition.cs	Scenario and network requirement schema.
Assets/Scripts/TssNetworkSessionState.cs	Runtime profile creation and derived seed-based values.
Assets/Scripts/TssNetworkWorkPackageEvaluator.cs	WP2-WP4 evaluation against authored requirements.
Assets/Scripts/TssExternalScenarioLoader.cs	Manifest loading, catalog resolution, scene availability, and scenario creation.
Assets/Scripts/TssExternalScenarioValidator.cs	Scenario and requirement validation.
Assets/Scripts/TssScenarioCatalog.cs	Catalog identity and scene compatibility validation.
Assets/Scripts/TssScenarioStartupMenu.cs	Selected scenario launch and scene transition.

sites/tss-scenario-config-editor/dist/app.js	Guided editor, validation, persistence, and download behavior.
Assets/StreamingAssets/Scenarios/manifest.json	Scenario-to-file, catalog, and scene registration.

Acceptance criteria

- Unity compiles with zero errors and zero warnings.
- The existing Capstone scenario loads and launches in the current TSS scene.
- A test scenario with a unique scene name can pass manifest, scenario, catalog, and Build Settings validation.
- A scenario referencing a missing scene is rejected before launch.
- A scenario whose scene and catalog do not match is rejected.
- The editor accepts, edits, preserves, and downloads a custom sceneName.
- The editor exposes the new evaluation requirement arrays and scalar flags.
- WP1-WP4 evaluation results continue to use scenario-derived runtime values.
- VLAN and IP generation remains deterministic and student-ID seed variation remains unchanged.
- Existing TSS fallback behavior and scene/prefab bindings remain intact.

Validation plan

Check	Evidence
Static JSON	ConvertFrom-Json succeeds for each scenario and manifest.
Editor validation	Guided editor reports the scenario ready for Unity validation.
Unity compilation	Unity console has zero errors and warnings.
Runtime profile	Logged profile confirms authored requirements and unchanged seed-derived VLAN/IP values.
Evaluator regression	Focused checks cover default Capstone requirements and a custom requirement variation.
Scene launch	Current TSS and a unique test scene both launch through the scenario menu.

Constraints and rollback

Do not remove the individual-tile map path, rewrite user-authored prefab layouts, or move scripts. Preserve serialized field names. If custom scene support is not ready for all catalogs, keep the existing TSS restriction temporarily and fail clearly rather than silently launching the wrong scene.

Rollback is limited to reverting the new scenario schema fields, profile copies, evaluator selectors, editor validation, and scene/catalog validation changes. Existing Capstone fields and the TSS baseline should remain usable throughout the work.

Implementation status: planning document. No custom scene or loader changes are authorized by this document alone.